Neurocomputing 390 (2020) 248–259

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Visual Recognition of traffic police gestures with convolutional pose machine and handcrafted features

Jian He[a], Cheng Zhang[a,*], Xinlin He[b], Ruihai Dong[c]

[a] Beijing University of Technology, Beijing, China
[b] University of British Columbia, Vancouver, Canada
[c] UCD School of Computer Science, Dublin, Ireland

ARTICLE INFO

Article history:
Received 29 November 2018
Revised 18 July 2019
Accepted 26 July 2019
Available online 19 October 2019

Keywords:
Gesture recognition
Human–machine interface
Traffic police gestures
Convolutional neural network
Deep learning

ABSTRACT

Autonomous vehicles have become a hot spot of the automotive industry, many cities have claimed that autonomous vehicles should be capable of recognizing gestures used by traffic police. Traditional traffic police gesture recognition methods rely on depth-sensor or wearable-devices, which limits their availability in the domain of the intelligent vehicle. Vision-based methods have fewer requirements for distance, but the modeling process is challenging due to the complexity of the visual scenes. Inspired by the recent success in vision-based pose estimation networks such as Convolutional Pose Machine (CPM), in this paper, we propose a novel vision-based human-machine interface to recognize eight kinds of Chinese traffic police gestures and apply it in the real-time recognition tasks. This method integrates a modified CPM network and two kinds of handcrafted features: Relative Bone Length and Angle with Gravity as spatial domain features, and adopt a Long short-term memory (LSTM) network to extract temporal domain features. To train and validate our method, we create a gestures dataset with two hours of traffic police gesture videos, which has 3354 gesture instances. The experiment results show that the proposed method is capable of recognizing traffic police gestures, and is fast enough for online gesture prediction.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

With the fast growth of urbanization in China, traveling by car has become more and more popular. Recognizing gestures used by authorized people, such as police officers, is one of the essential skills of drivers. For example, when there are traffic jams or traffic lights fail, traffic police usually controls traffic and guides drivers through their gestures. Facing the upcoming era of automatic driving, many cities have begun to develop regulations to regulate autonomous vehicles, and one of the important concerns is that auto-driving vehicles should be capable of recognizing these gestures and dealing with them within a limited time. Therefore, the development of traffic police gesture recognition methods is one of the essentials of building driver-assistance systems for intelligent vehicles.

The Chinese Traffic Police Gestures system (namely CTPGs) is defined and regulated by the Chinese Ministry of Public Security, of which the gestures are joint actions and movements of arms and torso. To recognize these gestures usually faces two challenges. Firstly, it is very difficult to extract skeletal features from traffic police images, since in an unpredictable and complex scene, the extracting process can be affected by many factors, such as police height and dress, light conditions and road conditions, etc. The second challenge is how to reduce or eliminate the effects of interferences during classification, such as absolute locations and distances. Fortunately, leveraging deep learning techniques, a pose estimation method called Convolutional Pose Machine (CPM) [1] manages to extract human part locations from a complex image, which provides a base for the RGB image skeletal feature extraction.

To address these issues and inspired by the work of [1], in this paper, we design a novel recognition method, which applies a modified CPM to extract skeletal data, then converts them with two kinds of handcrafted spatial features: Relative Bone Length and Angle with Gravity, and finally sends the spatial features into a Long short-term memory (LSTM) network [2] to recognize the class of traffic police gestures. The contributions of our work can be summarized as follows.

- First, we produce the Police Gesture Dataset, which contains the videos of Chinese traffic police commanding gestures, and ground truth gesture labels for each video frame.
- Second, we propose a method to extract two kinds of handcrafted features from human skeletal features, which acts as a

* Corresponding author.
E-mail address: 18618135402@163.com (C. Zhang).

https://doi.org/10.1016/j.neucom.2019.07.103
0925-2312/© 2019 Elsevier B.V. All rights reserved.

bridge between CPM and LSTM, and significantly improves the recognition accuracy.
- Third, we combine a modified version of CPM with handcrafted spatial features and LSTM to develop the Chinese traffic police gesture recognizer. Comparing with traditional depth sensor-based methods, this recognizer can work on RGB videos. Meanwhile, it is fast enough for real-time recognition tasks.

To prove the authenticity of our method, we have released our dataset and the implementation code for reproducibility[1]

2. Related works

Because of the importance of traffic police gesture recognition, a lot of studies have been carried out. These studies can mainly be divided into two categories: the wearable sensor based method, and the vision sensor based method. The wearable sensor based method uses inertial MEMS (micro electro-mechanical systems) sensors (e. g. accelerometers and gyroscopes) to measure the movements of police arms, and inferences the poses according to those movements. For instance, Wang and Yuan [3] applied two accelerometers on hands to classify the command gesture of the traffic police. Yuan and Wang [4] used digital gloves which wearable sensors were fixed on the back of the police hands to extract the gesture data. Although these methods can get a good recognition rate, and are capable of working in a long range, they require the police to wear the designated devices, which is an extra hindrance in practical situations. Additionally, the relatively high cost of the devices also limits its use in traffic police gesture recognition.

Vision sensor based methods mainly include two steps: The first step acquires the gesture video from a digital camera, then extracts spatial features from the video. The second step is to classify gesture based upon the spatial features. Vision sensor based methods have been widely used in gesture recognition, and achieved both scientific and economic success because of its convenience and relatively low cost. For example, Guo et al. [5,6] used a max-covering scheme to recognize gestures of Chinese traffic police. The scheme mainly based on the observation that body-part tiles maximally cover the foreground region and suit a body plan. Then the rotation joint angle or Gabor feature based two-dimensional (2D) principal component analysis (PCA) is used to recognize the command gestures. For this method, RGB camera is used to capture testing images from video, and a 5-part body model is constructed according to a 2D model. Therefore, it is difficult to deal with the situation when traffic police arms are perpendicular to the image plane. Eichner et al. [7,8] proposed a method to evaluate the spatial layout of humans (namely the position of the head, torso and arms) in static images. Once a person is identified by the upper-body detector, the method for searching their body parts can be simplified via the constraints on position and appearance according to that detection. Besides, Kang et al. [9] combined gesture spotting with gesture recognition, and classified 10 kinds of upper-body gestures as the interface between a video game (Quake II) and its users. Song et al. [10] presented an unified framework for body and hand tracking so as to understand simultaneously performed body-and-hand gestures. In addition, they introduced a multi-signal gesture database (namely, the NATOPS aircraft handling signals) including 24 body-and-hand gestures.

Being integrated with a depth sensor, Microsoft Kinect [11] can build 3D models for objects within 5 m, as well as extracting depth information, locating human skeletons and filtering backgrounds. It has been widely used on pose recognition. For example, Guo et al. [12] proposed a method that recognizes 8 kinds of traffic police gestures via Kinect based on static and dynamic descriptor fusion, and used a mean structural similarity index to recognize traffic police gestures. Le et al. [13] proposed a real-time traffic gesture recognition test platform system based on support vector machine (SVM) training data, in which traffic police command gestures are captured in the form of a depth image from a visual sensor, and the human skeleton is then constructed by a kinematic model. Zhou et al. [14] presented a real-time hand posture recognition method in color-depth images, and the histogram of gradient features of a hand posture was computed for Extreme Learning Machine classifier to recognize different postures. Though the integrated vision sensor helps to provide a more precise position of human parts, and improves the accuracy of gesture recognition. The working range of the depth sensor is limited, which results in the influence of the accuracy of gesture recognition when the distance between Kinect and traffic police is large.

The effectiveness of machine learning and deep learning has been demonstrated in many fields besides image classification, such as image retrieval [15] and location recommendation [16]. Similarly, studies on deep-learning based solutions for pose estimation and action recognition fields have multiplied for the past few years [17]. The pose estimation methods produce 2D or 3D locations of human parts. For example, Wei et al. [1] incorporated a convolutional network architecture into the pose machine framework so as to allow the learning of representations for both image and spatial relationships. This method provides a systematic approach to compose convolutional networks with large receptive fields for 2D pose estimation tasks. Cao et al. [18] used a convolutional neural network (CNNs) to learn the association (namely Part Affinity Fields, PAFs) of body parts, and designed an architecture to jointly learn 2D part locations and their association via two branches of the same sequential prediction process. The architecture encodes global context, allowing a greedy bottom-up parsing step that maintains high accuracy while achieving a real-time performance, irrespective of the number of people in the image. Mehta et al. [19] presented a single RGB camera-based method that yields temporally stable 3D global pose reconstructions using a convolutional network. They used the temporal history of estimation sequence as features to ensure smooth tracking of coordinates over time.

Action recognition tasks aim to recognize single or multiple actions occurred in a sequence of images. In recent years, deep learning solutions for action recognition tasks are widely studied. For example, Bilen et al. [20] summarize the dynamics of a entire video sequence as an image, and train a dynamic-aware network for action recognition. This method is powerful and efficient against fixed-length video sets with one action per video, such as UCF101 [21] and HMDB51 [22], but it can not be applied for datasets with multiple actions per video. Donahue et al. [23] present Long-term Recurrent Convolutional Networks (LRCNs), which extracts visual features from input by CNNs, then sends those features into an LSTM sequence to predict the actions. The advantage of LRCNs is that the prediction length is variable, which is suitable for handling arbitrarily long sequences. Zhang et al. [24] transfer the features in optical flow CNN to that of motion vectors CNN in each frame, and managed to achieve real-time performance in action recognition tasks. However, LRCNs and motion vectors CNN learn massive feature representations from images directly, which weakens the attention of the network to human.

Our method relies on extracting the skeletal data from monocular RGB images to separate the human from the background, so it is a skeleton based method. There are already many skeleton based methods for gesture recognition, this paragraph discusses the reason of not using them to recognize traffic police gestures. Zhu et al. [25] published a survey to describe the evolution of action recognition methods, which covers some skeleton-based approaches, but

[1] https://github.com/zc402/ChineseTrafficPolicePose

Table 1
Dataset splitting.

Set	Films	Gestures	Stand in attention
Training	11	1789	900
Test	10	1565	787
Total	21	3354	1687

none of them extracts the skeletal data from RGB image. For example, Wu and Shao [26] proposed a hierarchical parametric network to classify and segment the actions. This method is a 3D skeletal joint based method, but the skeletal data is provided by Kinect, a depth camera that is designed to be used within short ranges, which is not available for recognizing the police on road. Du et al. [27] proposed an action recognition method that divides the limbs into parts by human physical structure. This method shows great improvements on various datasets. However, the skeletal features used by this method comes from the public datasets. Two of the three datasets mentioned in the paper use depth sensor to collect the joint coordinates, and one uses LED marker which is counted as wearable sensor. According to the previous discussion, wearable sensors are also unavailable for police gesture recognitions. Liu et al. [28] presented a detailed survey on RGB-D sensor based methods with more than 30 proposes use skeletal data as their input, but all the data are collected from depth sensor. This article gives our research a good support by reviewing the properties of RGB-D sensors. According to the Table 1 in Liu's survey, only the most advanced deep sensor can work outdoor with a range limitation of 10 m. The Kinect, which is used by most of the gesture recognition methods above, can only work indoor within 4meters, which is clearly not enough for autonomous cars. Besides, the 3D poses predicted by depth sensors are extremely accurate and precise, while the 2D or 3D poses inferred from RGB images can be interfered by various factors such as occlusions, so the quality of output poses is different. Therefore, it is hard to directly move the RGB pose estimation methods onto the top of the depth sensor based methods to recognize the traffic police gestures. Aggarwal et al. [29] reviewed gesture recognition methods with 3 kinds of skeletal data sources: marker-based motion capture system, reconstructed 3D data from multiple views, depth sensors. Among them, the marker-based system requires the target to wear special suits, which is not realistic in practice. As for the multiple views reconstruction systems, according to relevant article [30], they still lack enough range to detect police on road. Therefore, although there are many methods aim to recognize gestures, few of them can be referenced considering the restrictions caused by the police gesture recognition task.

3. Methodology

This section introduces eight kinds of Chinese Traffic Police Gestures, then describes the extraction of the handcrafted spatial features in details, and explains the extraction of temporal features using the sequence of spatial features as input. Finally, for each frame, a gesture class is inferred from the temporal features.

3.1. Chinese traffic police gestures

The Chinese Traffic Police Gestures are composed by 8 kinds of gestures: (1) stop, (2) move straight, (3) left turn, (4) left turn waiting, (5) right turn, (6) lane changing, (7) slow down, (8) pull over. An extra gesture (0) "stand in attention" is used in this paper to describe the situation of which a police does not change the current traffic.

Fig. 1 illustrates actions of Chinese Traffic Police Gestures. These gestures are explained as follow: (1) To express "stop", the police faces the vehicle and raises his left arm. (2) To express "move straight", the police stands laterally to the traffic, extends his both arms, then bends his right arm. (3) To express "left turn", the police raises his right arm, then swings his left arm from side to center of body. (4) To express "left turn waiting", the police stands laterally to the traffic, then swings his left arm up and down. (5) To express "right turn", the police raises his left arm, then swings his right arm from side to center of body. (6) To express "lane changing", the police extends his right arm, parallel to the right shoulder, then swings to the left. (7) To express "slow down", the police extends his right arm, parallel to the right shoulder, then swings down. (8) To express "pull over", the police extends his right arm, parallel to the waist, then raises left arm, and swing right arm to the left.

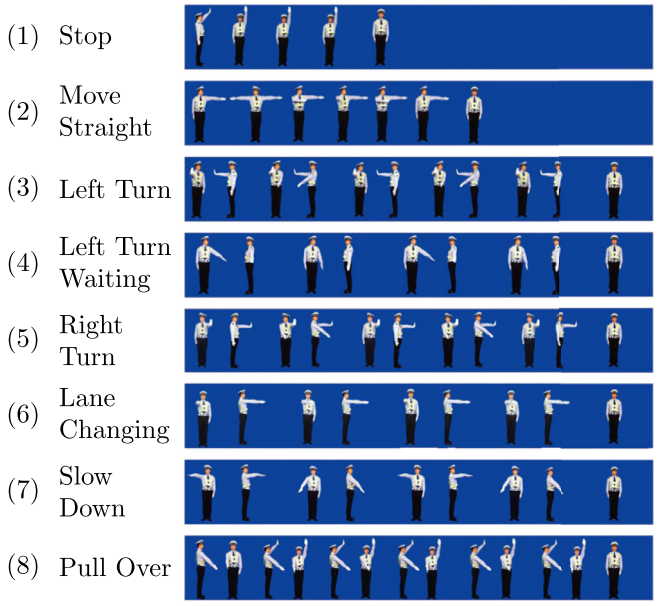


Fig. 1. Actions for Chinese traffic police gestures

3.2. Extraction of spatial features

The overall process of spatial feature extraction from input image to spatial features is shown in Fig. 2. Fig. 2 includes 5 subfigures connected with arrows above them. Arrows denote the processes of handling spatial features, and subfigures denote the outputs of these processes. Subfigure (a) shows an input image frame with a police making gestures, the small yellow dots denote the parts of this police. The part in this article refers to the anatomical landmarks of a person, it's also called joint or keypoint according to different papers. Subfigure (b) illustrates the absolute coordinate position of the parts by big yellow dots. These dots are collectively known as the articulated pose of the person, which is the collection of the absolute part positions on the image. There are totally P part positions, marked as Y_p. The arrow between (a) and (b), marked as process g_T, is used to predict articulated pose of the police from the RGB image. In this article, g_T is implemented as a convolutional network modified from Convolutional Pose Machine (CPM) [1]. Subfigure (c) illustrates the Bone Vectors. In this figure, the parts (yellow dots) are connected by green arrows with mark S_c. This paper defines the pattern of how these parts are connected with each others as Connection Dependency. According to the Connection Dependency, there are totally C connections between parts, each of the connection $c \in \{1, ..., C\}$ contains 2 parts, namely starting part m and ending part n, $m, n \in P$. Based on the Connection Dependency, the Bone Vectors S are defined as vectors pointing

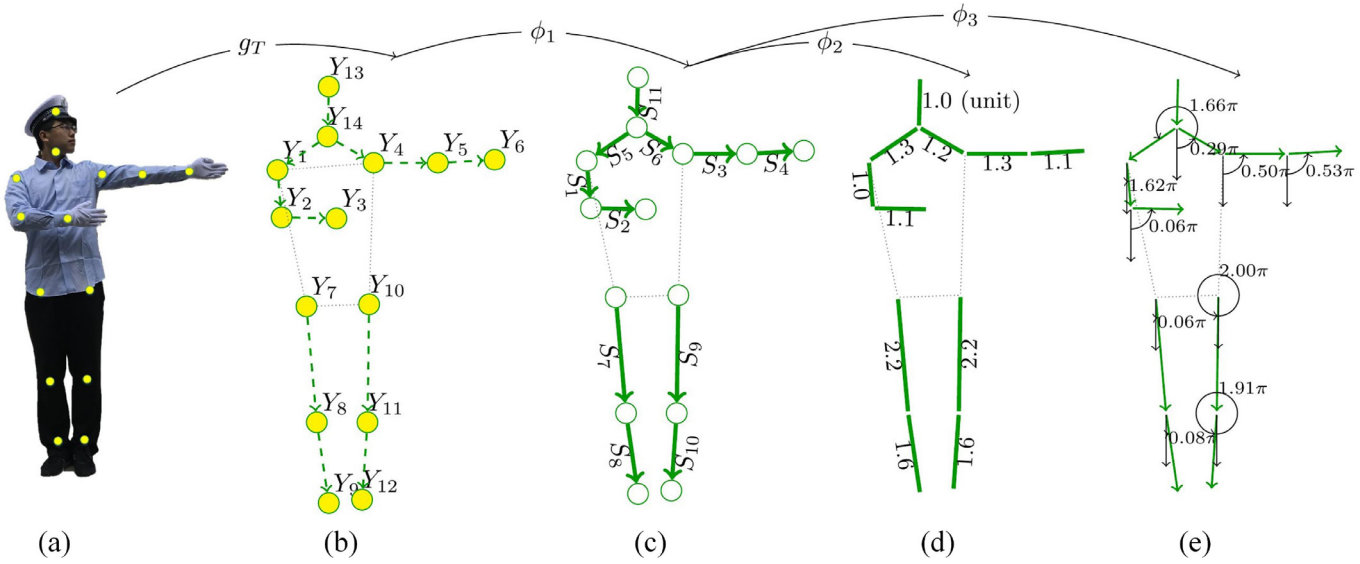


Fig. 2. Spatial feature extraction process.

from starting part m to ending part n and illustrated as green arrows in subfigure (c). Like any other vectors, the Bone Vectors only contain lengths and directions, and do not concern about absolute coordinate positions of parts. Process ϕ_1 showing above the subfigure (b) and (c) is used to convert part positions to Bone Vectors. Subfigure (d) illustrates the Relative Bone Length. This figure shows some line segments with numbers attached, and a line segment marked as unit. The line segments refer to the length of Bone Vectors, and the unit line segment refers to the Bone Vector of head. The numbers attached shows the relative length of Bone Vectors with respect to head vector. These length numbers are the first kind of spatial features, collectively called Relative Bone Length F_1. Process ϕ_2 is used to convert Bone Vectors set S to Relative Bone Length F_1. Subfigure (e) illustrates the Angle with Gravity. This figure shows two kinds of arrows with different colors: green arrows and black arrows. The green arrows are pointing to the same directions with Bone Vectors, the black arrows are pointing vertically downward. Each pair of green and black arrow shares a starting point, and the angles between them are marked by radians. The green arrows represent the directions of Bone Vectors, and the black arrows represent the direction of the gravity. The angles between them are the second kind of spatial features, collectively called Angle with Gravity F_2. Process ϕ_3 is used to convert Bone Vectors set S to Angle with Gravity set F_2. At last, the spatial feature set F is the the concatenation of Relative Bone Length F_1 and Angle with Gravity F_2.

To implement the first process g_T in Fig. 2, a modified Convolutional Pose Machine (CPM) network is applied. The input of CPM is the set of colors from each position (u, v) of an image, denoted by Z, $Z \subset \mathbb{R}^2$. The pixel position of each part in the image is denoted by Y_p, $Y_p \in Z$. A person have P parts in total, the position of parts in the image $(Y_1, ..., Y_p)$ is denoted by Y. $g_t(\cdot)$ is a classifier, which is used to predict the confidence of a point z belonging to part Y_p. The value of confidence is denoted by $b(Y_p = z)$, $z \in Z$. The subscript of the classifier $t \in 0, ..., T$ denotes the stage of pose estimation. Each stage $g_t(\cdot)$ can be seen as an independent classifier, with different convolutional architectures and identical losses. Classifier of each stage gains a larger receptive field than that of previous stage, and outputs a refined prediction. At the 1st stage, when $t = 1$, classifier produces following confidence map:

$$g_1(x_z) \rightarrow \{b^p(Y_p = z)\}_{p \in \{1,...,2P\}} \tag{1}$$

In Eq. (1), as the input, x_z denotes the features at image location z. $b^p(Y_p = z)$ denotes the confidence score map of location z belonging to part p. The confidence score map can be seen as an array with each element representing the evidence score of a pixel in the image belonging to a human part, or belonging to the background. P denotes the total number of parts, the set $\{1, ..., 2P\}$ denotes P confidence score maps of a pixel belonging to a part, and P confidence score maps of a pixel not belonging to a part.

Following paragraph describes our modification on the CPM. In the original CPM paper, the network only produces $P + 1$ confidence score maps for P parts and 1 background (namely style (a)), so the confidence score map set is $\{b^p(Y_p = z)\}_{p \in \{0,...,P\}}$, in which the b^0 represents the background scores. In their later work [18] on CVPR, the PAFs network increases the number of maps to $2 \times P$ for P parts and P backgrounds (namely style (b)). The difference is that in style (a), each part score competes with the scores of other parts and the score of the background to decide the probability of whether a pixel belongs to this part, while in style (b), since each part has it's own background score, it only competes with it's own background to produce a probability, and will not be affected by the score of other parts. In style (a), the probabilities of a pixel belonging to each part and the background collectively add up to one, while in style (b), the probabilities of each two scores: the score of a pixel belonging to part p and the score of this pixel not belonging to part p collectively add up to one. Style (b) takes advantage when multiple parts are close or overlapping, and the most obvious part gains extremely high scores around it's area. In this case, if style (a) is applied, then the score of this obvious part will suppress other parts around it's area, so only this one particular part will be found. However, if style (b) is applied, then the scores will not be affected by each others, so the occluded parts may still able to be found as long as their confidences are larger than the global threshold. In this article, we modify the Convolutional Pose Machine network to make it compatible with the network style (b).

Let w and h be the width and height of input image, and the confidence set of part p belonging to each location (u, v) in the image be $\mathbf{b}_t^p \in \mathbb{R}^{w \times h}$, then the $\mathbf{b}_t^p$ is denoted by:

$$\mathbf{b}_t^p[u, v] = b_t^p(Y_p = Z) \tag{2}$$

For convenience, the confidence set of each part is collectively denoted as $\mathbf{b}_t \in \mathbb{R}^{w \times h \times P}$, $\mathbf{b}_t(Y_p = Z) = \oplus_{i=1}^{P} b_t^i(Y_i = Z)$. The sign $\oplus$ denotes an operator for vector concatenation.

In subsequent stages where $t > 1$, classifiers still predict the confidence for $Y_p = z, \forall z \in Z$. The inputs are concatenation of 2 kinds of information: (1) image features x_z, (2) prediction results $\mathbf{b}_{t-1}$ from previous stage:

$$g_t(x_z, \mathbf{b}_{t-1}) \rightarrow \{b^p(Y_p = z)\}_{p\in\{1,...,P\}} \tag{3}$$

In Eq. (3), the CPM network produces confidence maps corresponding to human parts. Theoretically, when there is only one person in the image, after T stages, the highest confidence value in the confidence map ($argmax(\mathbf{b}_T^p)$) should be evaluated as the location of the corresponding part. However, in practical situations, parts are not always visible. In order to deal with the occlusion of the parts, a threshold variable δ_p is introduced to assign minus values to the invisible parts. The part positions are evaluated as Eq. (4).

$$Y_p = \begin{cases} argmax(\mathbf{b}_T^p), \forall p \in P & if\ max(\mathbf{b}_T^p) > \delta_p \\ (-1, -1) & if\ max(\mathbf{b}_T^p) < \delta_p \end{cases} \tag{4}$$

The process g_T is finished, it's output: evaluated part positions are absolute coordinates shown in Fig. 2(b). Next process is ϕ_1, which converts the coordinates to Bone Vectors, as shown in Fig. 2(c). Each Bone Vector is composed by two parts: starting part m and ending part n, denoted by Y_m^c and Y_n^c. Set $S(S_1, ..., S_C)$ denotes the set of Bone Vectors, which is computed by caululating the difference between Y_m^c and Y_n^c:

$$S_c = Y_n^c - Y_m^c \quad c \in C \tag{5}$$

In Eq. (5), C is the Connection Dependency, a set that contains the pairs of connected parts. In practical situations, some parts will be occluded completely, and can not be found by CPM. In a Bone Vector, if at least one part can not be found, then this vector is treated as if two parts are overlapping. The details are explained as follow:

$$\phi_1(Y, c) \rightarrow \begin{cases} S_c & if\ POS(Y_m^c)\ and\ POS(Y_n^c) \\ (0, 0) & if\ NEG(Y_m^c)\ or\ NEG(Y_n^c) \end{cases} c \in \{1, ..., C\} \tag{6}$$

In Eq. (6), Y is the set of part positions, C is the set of part connections, and $c \in \{0, ..., C\}$ is one of the connection. Each connection c contains two parts, namely starting part m and ending part n, $m, n \in P$. The location of m and n are denoted as Y_m^c and Y_n^c. $POS(\cdot)$ means that the part location is bigger than or equal to zero, denotes that the part is visible. $NEG(\cdot)$ means that the part location is less than zero, denotes that the part is occluded. $POS(Y_m^c)$ *and* $POS(Y_n^c)$ indicates that both parts in connection c are visible, $NEG(Y_m^c)$ *or* $NEG(Y_n^c)$ indicates that at least one part in connection c is occluded. According to Eq. (6), if one or both parts are occluded, they are treated as overlapping parts. This process finally produces the Bone Vector set shown in Fig. 2(c).

Based on the Bone Vector set S, two kinds of spatial features are extracted. The first one is Relative Bone Length F_1, as shown in Fig. 2(d). The second one is Angle with Gravity F_2, as shown in Fig. 2(e). The processes to extract them from Bone Vector set are denoted by $\phi 2$ and $\phi 3$.

To calculate Relative Bone Length F_1, following equation computes the length of each Bone Vector, then normalizes them by the length of head Bone Vector.

$$\phi_2(S) \rightarrow \left\{ \frac{||S_i||}{||S_{head}||} \right\}_{i\in\{1,...,C\}} \tag{7}$$

In Eq. (7), $S_{head} \in S$ denotes the Head Bone Vector which starts from head top and ends at neck. $||\cdot||$ denotes the norm, or the length of the vector. $\phi_2(\cdot)$ uses S_{head} as a reference, and produces the length of each Bone Vector with respect to the length of head. the output spatial features are shown in 2(d).

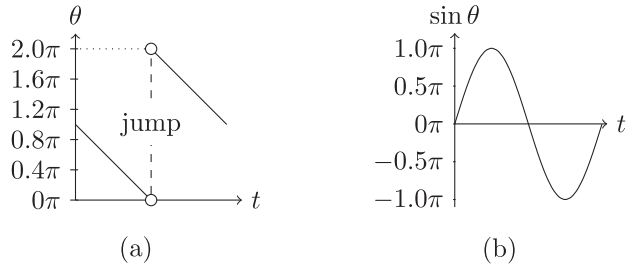


Fig. 3. The continuity differences between radian value and trigonometric value.

Process $\phi_3(\cdot)$ calculates the direction of Bone Vectors with respect to the direction of gravity, and produces spatial feature set F_2: Angle with Gravity:

$$\phi_3(S) \rightarrow \left\{ \frac{S_i \cdot d}{||S_i||\ ||d||}, \frac{S_i \times d}{||S_i||\ ||d||} \right\}_{i\in\{1,...,C\}} \tag{8}$$

In Eq. (8), d is the gravity unit vector, which points to the direction of gravity. $\frac{S_i \cdot d}{||S_i||||d||}$ produces the cos value of the angle between Bone Vector and gravity, and $\frac{S_i \times d}{||S_i||||d||}$ produces the sin value. To express an angle, the radian can be used, as shown in 2(e). However, the value of radian jumps between 2π and 0 when the person's arm go across the direction of gravity, so it is not the best choice to have radians represent the human actions. Therefore, to ensure the continuity of spatial feature values, the trigonometric values, instead of radians, are used to express the angle between the Bone Vectors and gravity, as shown in Eq. (8). The continuity differences between θ and $sin\theta$ are shown in Fig. 3.

Fig. 3 shows two plots, namely plot (a) and (b). plot (a) illustrates the radian value of angle when a part is rotated for 360 degrees, plot (b) illustrates the sine value. It can be seen that the radian value jump between the highest point and the lowest point, while the trigonometric value keeps the continuity. Therefore, the sine values and cosine values of Bone Vectors are used as spatial features.

At last, process ϕ_4 concatenates F_1 and F_2 and produces final spatial feature set for the model:

$$\phi_4(F_1, F_2) \rightarrow F_1 \oplus F_2 \tag{9}$$

3.3. Extraction of temporal features

Gestures consist of temporally consecutive actions, ignoring the temporal consistency may cause false classifications. Fig. 4 gives an example of two actions with similar spatial features belonging to different gestures, which requires the recognition method to further classify the temporal features. This figure compares the spatial features of two gestures (namely Stop and Pull Over), each gesture contains two consecutive actions. The RGB images at the bottom show the input frame, the black background images at the top show the Bone Vectors of two gestures. It is clear that the first actions of two gestures possess similar spatial features, and may confuse the classifier if the spatial features were the only available evidence. In this case, the best way to distinguish these two gestures is to infer the current gesture by looking at the spatial features from the past. In this article, the substantial evidences deduced from the spatial features of previous times are denoted by Temporal Features. Since each frame includes some new spatial features into the substantial evidences, their temporal features are always different with each others. These temporal features is ultimately used to decide the class of current gesture.

The temporal features of traffic police gestures depend on each of the spatial features produced in the past. A recurrent network, namely Long short-term memory [2], is able to extract temporal

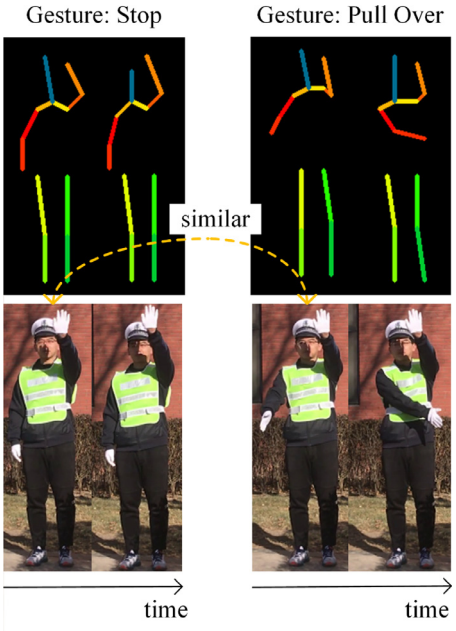


Fig. 4. Two actions with similar spatial features belonging to different gestures.

feature set h_τ from spatial feature set and save it's memory (cell states) at e_τ for the subsequent predictions. The e_τ is denoted by:

$$\begin{aligned} e_\tau = &\sigma(W_1 \cdot (h_{\tau-1} \oplus F^\tau) + \beta_1) * e_{\tau-1} \\ &+ \sigma(W_2 \cdot (h_{\tau-1} \oplus F^\tau) + \beta_2) \\ &* tanh(W_3 \cdot (h_{\tau-1} \oplus F^\tau) + \beta_3) \end{aligned} \tag{10}$$

While producing the e_τ, LSTM also produces h_τ as the temporal feature set which contains substaintial evidences for gesture prediction:

$$h_\tau = \sigma(W_4 \cdot (h_{\tau-1} \oplus F^\tau) + \beta_4) * tanh(e_\tau) \tag{11}$$

In Eqs. (10) and (11), $\sigma(\cdot)$ denotes the sigmoid function, $tanh(\cdot)$ denotes the hyperbolic tangent function. $\oplus$ denotes the concatenation operation, $\cdot$ denotes matrix multiplication, $*$ denotes pointwise multiplication. Since t was already registered as stage, a new notation τ is established to denotes time. F^t denotes the spatial features for police gestures produced at time τ. W and β denotes two trainable layers in neural networks: weight and bias accordingly. Lastly, a dense layer is applied to convert the temporal feature set h_τ to the prediction results:

$$o_\tau^{all} = s(W_5 \cdot h_\tau + \beta_5) \tag{12}$$

$$o_\tau^{max} = \begin{cases} argmax \quad (o_\tau^{all}) & if\ max(o_\tau^{all}) > \delta_g \\ 0 & if\ max(o_\tau^{all}) < \delta_g \end{cases} \tag{13}$$

In Eqs. (12) and (13), $s(\cdot)$ denotes softmax function, o_τ^{all} denotes the probability set of current gesture belonging to each gesture labels, o_τ^{max} denotes the final gesture label prediction. δ_g denotes the confidence threshold for gestures. When the confidence of each gestures is all smaller than the threshold, then no matter what class the $argmax(o_\tau^{all})$ shows, the class of "stand in attention" is produced to prevent false detections.

4. Network architectures

This section describes the architectures of the spatial feature extractor: modified CPM, and temperal feature extractor: LSTM in details.

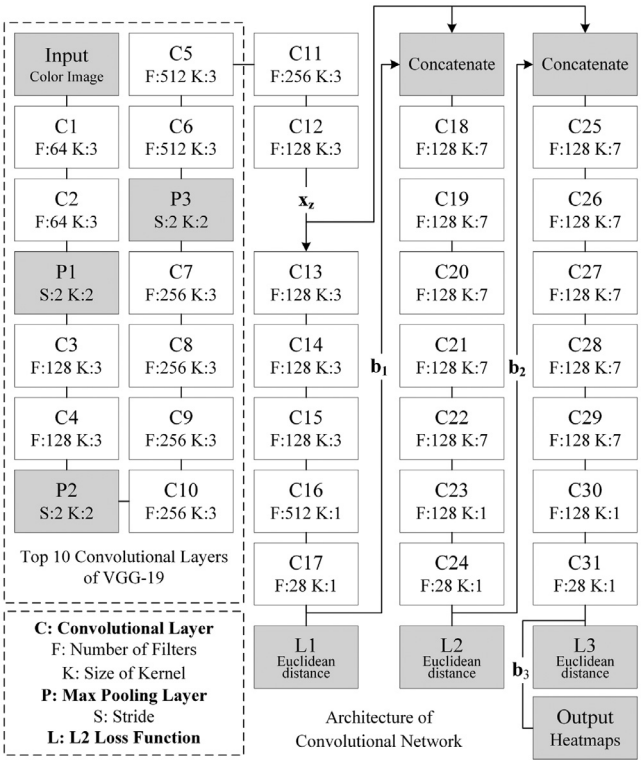


Fig. 5. The architecture of PKEN.

4.1. Convolutional network for part localization

To localize the human parts for spatial feature extraction, we introduce the Convolutional Pose Machine (CPM) and modified it's architecture. In details, we increase the number of score maps to acquire better performence when parts are overlapping, and decrease the stages to make the execution time shorter. To distinguish our modified architecture from original CPM, we reference our architecture as Police Keypoint Extraction Network (PKEN) in the following sections.

The architecture of PKEN is illustrated in Fig. 5. In the figure, C represents a convolutional layer, P represents a max pooling layer, L represents the loss. The top 10 convolutional layers of the network are the same as that of the VGG-19. The top 12 convolutional layers implements $\psi(\cdot)$, which extract image features from original image. There are 3 pooling layers among these convolutional layers, which means that the extracted image features are 8 times smaller in width and height than the original image. The 13th to 17th convolutional layers implement $g_1(\cdot)$, the stage 1 of CPM. It takes x_z as input, and outputs confidence set $\mathbf{b}_1$. The 18th to 24th convolutional layers implement $g_2(\cdot)$, the stage 2 of classifier, which accepts two inputs: x_z and $\mathbf{b}_1$ as input, and produces $\mathbf{b}_2$. Similarly, The 25th to 31th convolutional layers implement $g_3(\cdot)$. Fig. 6 illustrates the image of $\mathbf{b}_3$. Note that this image superimposes confidence maps for each part together. The total stage T is set to 3 in PKEN.

There are three losses in the network, namely L1, L2, and L3, which are the Euclidian distance of $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ with the ground truth S^*. Such architecture with multiple losses has been proved to be able to prevent the vanishing gradients problem during training [31].

Overall loss value of the whole network can be calculated according to the following Equation:

$$f = \sum_{p=1}^{P} \sum_{t=1}^{T} \sum_{\mathbf{z}} ||\mathbf{b}_t^p(z) - \mathbf{b}_*^p(z)|| \tag{14}$$

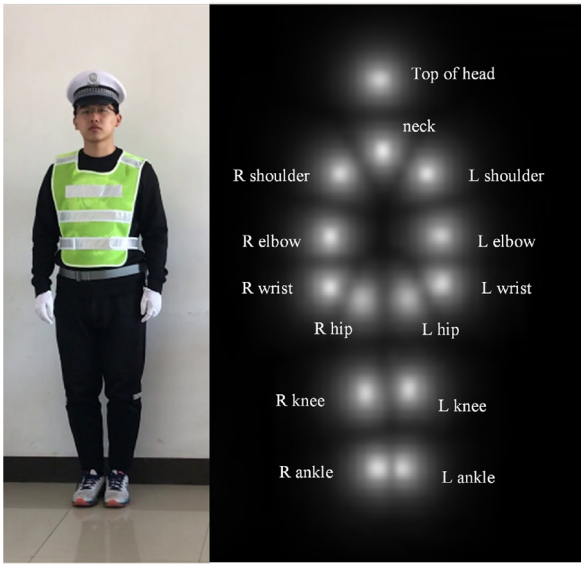


Fig. 6. Confidence maps for each part.

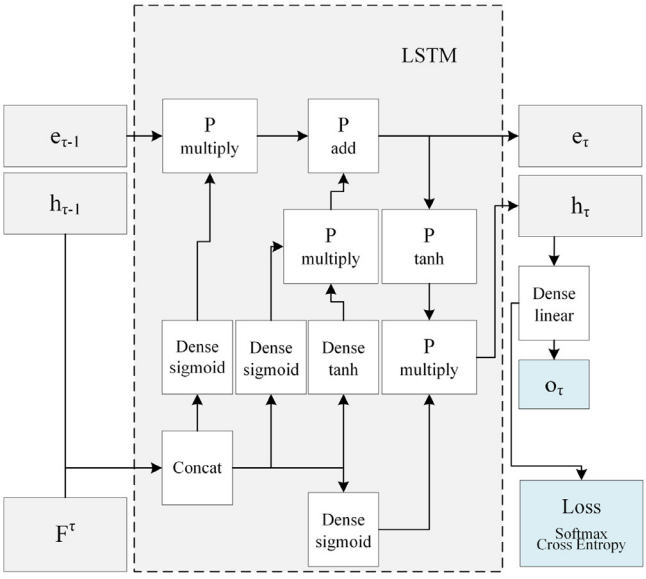


Fig. 7. Architecture of LSTM network.

In Eq. (14), $\mathbf{b}_*^p$ is the ground truth confidence map of part p, and z is the image location on the confidence map. Eq. (14) calculates the sum of Euclidean distance for each pixel of confidence map w.r.t. ground truth.

4.2. LSTM for temporal feature extraction

To extract the temporal features for police gesture recognition, Long short-term memory (LSTM) [2], a recurrent neural network, is applied. Fig. 7 illustrates the architecture of the LSTM used in our method.

In Fig. 7, $e_{\tau-1}$, $h_{\tau-1}$ and F_{τ} denote the inputs of the LSTM, and e_{τ}, h_{τ} denote the outputs. The notation "Dense" means fully connected layer, P denotes point-wise calculation. A dropout layer [32] is placed before the loss layer to prevent overfitting during training.

Let the ground truth label be o_{τ}^*, loss function for the network at time τ is:

$$f = crossentropy(o_{\tau}^{q}ll, onehot(o_{\tau}^*)) \tag{15}$$

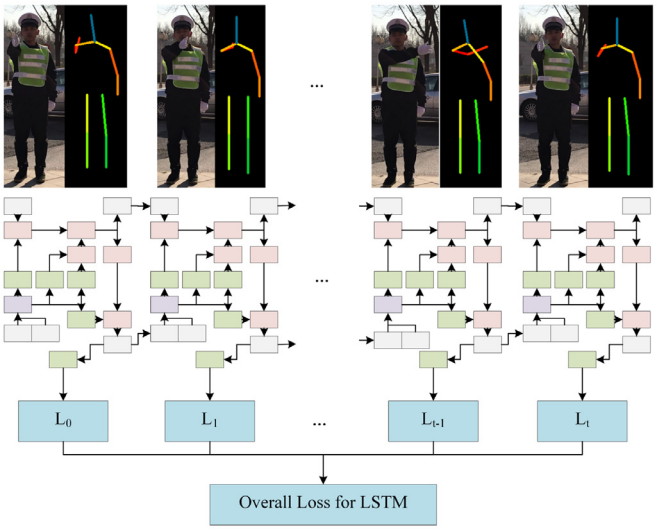


Fig. 8. Loss function of LSTM.

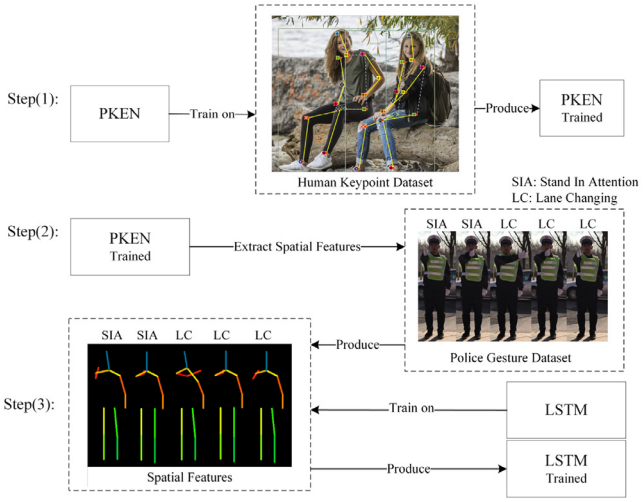


Fig. 9. Three steps training.

As a recurrent network, for each step of gradient descent, LSTM has to consider the loss of each frame for a certain duration, as shown in Fig. 8

The overall loss for LSTM is the mean of the loss at each time step. This method of training is called truncated backpropagation. Let the loss of LSTM at time τ be L_{τ}, current time be τ, then the overall loss is shown in Eq. (16).

$$f = \frac{\sum_{i=0}^{\tau} L_i}{\tau} \tag{16}$$

5. Datasets and training steps

This section describes the training details of PKEN and LSTM, introduces datasets, training orders and preprocessing steps. PKEN and LSTM are trained on two datasets respectively. The training process has three ordered steps which are illustrated in Fig. 9.

5.1. Datasets

As shown in Fig. 9, two datasets are involved during the training process.

The first dataset is a human keypoint dataset containing the coordinates of human joints such as wrist, shoulder, and neck, etc. In

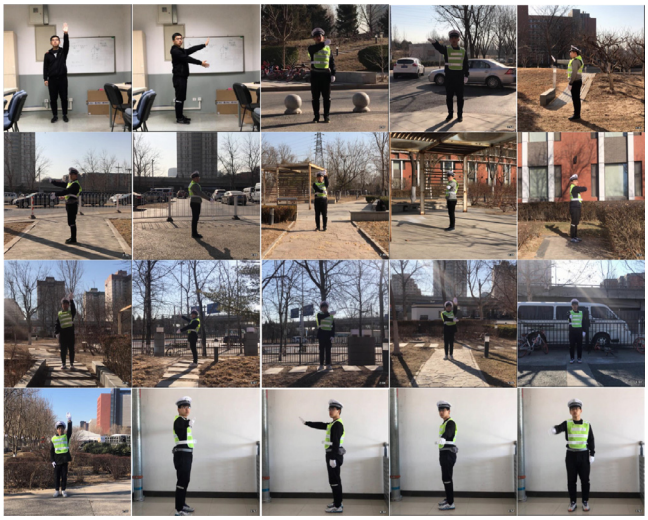


Fig. 10. Police Gesture Dataset video gallery. This gallery shows one picture per video, arranged from left to right: 001, 002, 003, 004, 007, 008, 009, 010, 011, 012, 013, 014, 015, 016, 017, 018, 101, 102, 103, 104.



Fig. 11. Time delay of the labels.

this paper, the dataset provided by AI Challenger [33] is applied on training.

The second dataset is the Police Gesture Dataset presented by us. The Police Gesture Dataset contains 20 videos of people making traffic police gestures and is annotated frame by frame. The dataset has a total video length of approximately 2 h showing different people and various backgrounds. A gallery of these videos is shown in Fig. 10, and the splitting of the dataset is shown in Table 1.

In Table 1, the dataset is divided into 2 subsets for training and test purpose. In the table, "Set" indicates the name of subsets."Films" indicates the number of files in a set. "Gestures" indicates the total number of gestures occured in a set. "stand in attention" indicates the number of stand in attention gesture. The films is shot with different devices in different scenes, and people with different appearance and police attire. The camera distances are also varied.

5.2. Label preprocessing

Before training, the appearance time of each label in Police Gesture Dataset is delayed for 750 ms after the occurrence of gestures. This time delay is essential for the entire pipeline to work properly, and dramatically increases its accuracy.

Fig. 11 illustrates how labels are delayed. the figure illustrated a few frames clipped from the dataset, with annotated labels. There are three rows named image, original labels and delayed labels. The original labels and the delayed labels use gray area to express one kind of gesture: Stand In Attention, and use green area to express another kind of gesture: Pull Over. Originally, the Pull Over label appears as soon as the police raises his hand. After delaying the labels, the Stand In Attention area becomes longer, which means that the appearances of Pull Over labels are delayed. The reason of delaying the training labels is: originally, a label appears as soon as the gesture occurred. If the original labels are used to train the network, it means that the network has to response immediately as soon as a gesture occurred, which makes the network over-sensitive against each small movement of the police, and makes wrong predictions frequently. Therefore, the timing of labels are delayed to leave some time for the LSTM to observe, so the network will not judge the meaning of the gesture until the police made enough movements. It turns out that delaying the appearance of labels significantly improved the accuracy of the network, and also made it robust against small movements that is not related to any gestures. At the cost, delayed training labels decrease the method response speed. A comparison of performances between human and our method trained with delayed labels is given at the results section.

5.3. Training steps

As shown in Fig. 9, our model, the Traffic Police Gesture Recognizer, is not an end-to-end network that can be trained directly. three steps are required to finish the training. In step (1) PKEN is trained on keypoint dataset. In step (2) spatial features are extracted from Police Gesture Dataset using trained PKEN. In step (3) the LSTM is trained on spatial features.

The overall process of training is shown in Algorithm 1. In the algorithm block, KP_I denotes the total number of images contained in Human Keypoint dataset, and KP_i denotes the image number i. PG_I denotes the total number of frames contained in Police Gesture dataset, and PG_i denotes the ith frame. F^i denotes the spatial feature F of ith frame. PG_L denotes the total number of frames contained in spatial features, F^{clip} denotes a random clip created from F^τ.

Algorithm 1 Train CTPGR

```
Initialize PKEN weights
for i = 1...KP_I do
    for p = 1...P do
        Read an image KP_i from keypoint dataset
        Create b*^p from KP_i using Eq. (17)
    Train b^p using b*^p
PKEN predictor b^p trained
for i = 1...PG_I do
    Read a frame PG_i from police gesture dataset
    Generate F^i from PG_i by predictor b^p
Spatial features F^τ_{τ∈0,...,PG_L} generated
Initialize LSTM weights
while LSTM loss decreasing do
    Create a random clip F^clip from F^τ_{τ∈0,...,PG_L}
    Train LSTM by F^clip
LSTM trained
Build CTPGR by connecting PKEN and LSTM
```

To train the PKEN in step (1), the annotated human keypoints are converted to ground truth confidence maps. A 2D Gaussian function is used to convert annotated points to ground truth of confidence map, which is shown in Fig. 12.

Let K be the number of people, ground truth confidence map for part p can be calculated according to following Equation:

$$b_*^p(z) = \sum_{k=1}^{K} exp\left(-\frac{||z - n_{p,k}||}{\sigma^2}\right) \tag{17}$$

Table 2
Edit distance between predicted labels and ground truth labels.

Filename	Predicted	N (True Gestures)	S (Substitution)	D (Deletion)	I (Insertion)	Accuracy
014.mp4	200	165	6	37	2	0.7273
016.mp4	186	161	2	27	2	0.8075
004.mp4	99	100	0	0	1	0.9900
018.mp4	165	163	1	4	2	0.9571
010.mp4	172	145	1	27	0	0.8069
104.mp4	161	167	0	0	6	0.9641
102.mp4	164	165	0	3	4	0.9576
012.mp4	179	177	0	2	0	0.9887
008.mp4	159	161	0	0	2	0.9876
002.mp4	170	161	0	9	0	0.9441
AVERAGE	165.5	156.5	1	10.9	1.9	0.9130
TOTAL	1655	1565	10	109	19	0.9118

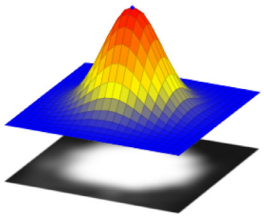


Fig. 12. 2D Gaussian function for making ground truth map from annotated points.

Table 3
Architecture comparison.

No.	Architecture	Reference	Accuracy
1	(Ours) PKEN + LSTM		91.18%
2	PKEN + Bidirectional LSTM	Pigou et. al. [34]	91.04%
3	Resnet Part Localizer + LSTM	He et. al. [35]	87.29%
4	Densenet Part Localizer + LSTM	Huang et. al. [36]	89.74%
5	End to end 3D Convolution	Ji et. al. [37]	81.16%
6	Convolutional LSTM	Xing et. al. [38]	82.40%

In Eq. (17), n is the annotated points, σ is used to control how much the Gaussian peak spreads.

In step (1), to establish the PKEN network, we build the network in Fig. 5 with rectified linear unit (ReLU) activation function, and initialize all the neurons with "he initializer" [39]. As for training, we choose a batch size of 15, and adopt Adam optimizer [40] for gradient descent, with a learning rate of 0.0008 and an exponential decaying rate of 0.8 for every 20,000 steps. Data augmentation methods such as rotation and scaling are applied to increase the variety of dataset. We train the PKEN for 50 epochs.

In step (2), we run the forward pass of the trained PKEN to extract part location from Police Gesture Dataset into spatial feature F. PKEN parses the police gesture videos frame-by-frame, generates a sequence of spatial features. The temporal labels marked for the Police Gesture Dataset is unchanged and used to mark ground truth gestures for these spatial features.

In step (3), to establish the LSTM network, we build the network in Fig. 7, and initialize all the neurons with Xavier initializer [41]. No data augmentation method is applied. As for training, we randomly clip the sequence of spatial features into smaller fragments. Each of the fragment is a video with 90 seconds duration, and 128 fragments compose a batch. We adopt Adam optimizer for truncated back propagation, with a learning rate of 0.0004, and train the network for 50,000 steps.

After the training, feature extractor and classifier are connected in order to form the Traffic Police Gesture Recognizer. The entire network takes a sequence of images as input, and outputs a sequence of gesture predictions.

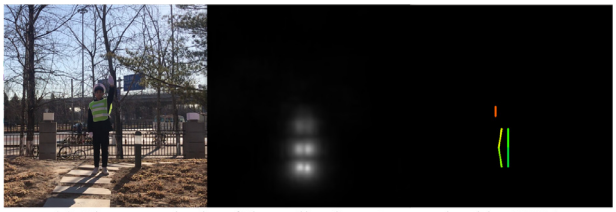
6. Evaluation results

The Accuracy of Traffic Police Gesture Recognizer is evaluated on the test set of the Police Gesture Dataset. The offline evaluation result is shown in Table 2. The metrics for this table is called Edit Distance, which is a common metrics [42] to evaluate the accuracy of time series labels. This metrics contains 4 notations: *N, I, D, S*. Notation N indicates the total number of ground truth gestures in the video. In our case, since the method needs to distinguish the state of "stand in attention" from the video, each "stand in attention" gesture is also counted as N. Notation I indicates the number of insertions from predicted labels to ground truth labels, $I = i$ means that if i characters were inserted to the designated places of predicted labels, then predicted labels become same as ground truth labels. D indicates the number of deletion from predicted labels to ground truth labels. $D = d$ means that if d characters were deleted from predicted labels, then the predicted labels and ground truth labels become equal. S indicates total number of substitution, $S = s$ means that if s characters were substituted from predicted labels, then predicted and ground truth label series become equal. Finally, the value of accuracy is produced as: $Accuracy = (N - I - D - S)/N$. For completeness, extra information are presented. "Filename" indicates the corresponding file in the dataset. "Predicted" indicates the number of predicted gestures. The "AVERAGE" row shows the average of above rows. The "TOTAL" row sums the above rows then computes the accuracy, which is equal to concatenating all the films together.

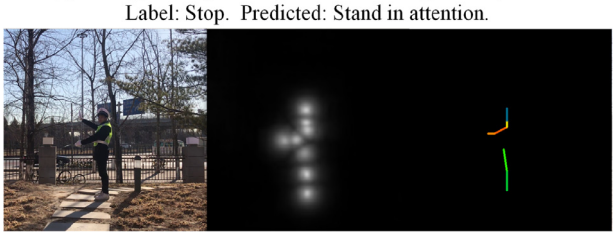
The results in Table 2 shows that the film "014.mp4" has the lowest accuracy among all the test films. We analyzed this film, and found that at some frames, the CPM network failed to recognize the parts of police, as shown in Fig. 13. Figure (a) shows a bad case where the CPM failed to recognize most of the parts, and the LSTM produced a wrong result. Figure (b) shows a bad case where the CPM failed to recognize the right arm of the police. Fortunately, the recognized parts (body and left arm) provides enough evidence for the LSTM to predict a correct label. Figure (c) shows a normal case from 012.mp4. In this frame, the CPM correctly recognized all the parts, and LSTM predicted a correct label.

A real-time recognition test (online test) is performed to validate the online availability of our method. To acquire a stable result, we performed the online test 4 times in different places. The image acquiring speed for a real-time scene is 65 ms per image, so the frame rate keeps around 15 fps. The accuracy of real-time recognition is shown in Table 4. In the table, "Number" indicates the sequence number of a test, and other notations are the same with that of the Table 2. The accuracy of our method under online scene shows almost no different with the offline test according to Table 4.

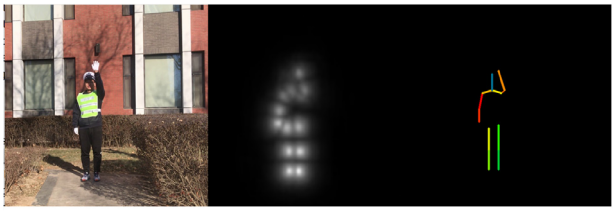
The total stage number T in CPM controls how many times the confidence maps are refined before producing the result.



(a) The upper body of the police is not recognized by CPM.
Label: Stop. Predicted: Stand in attention.



(b) The right arm of the police is not recognized by CPM
Label: Left Turn. Predicted: Left Turn.



(c) A correctly recognized frame
Label: Stop. Predicted: Stop

Fig. 13. Badcases where the CPM network failed to recognize parts

Table 4
Accuracy of online test.

Number	Pred	N	S	D	I	Accuracy
No.1	79	80	2	1	2	0.9375
No.2	67	62	2	5	0	0.8870
No.3	66	65	0	2	1	0.9538
No.4	123	121	1	4	2	0.9421
AVERAGE	83.75	82	1.25	3	1.25	0.9301
TOTAL	335	328	5	12	5	0.9329

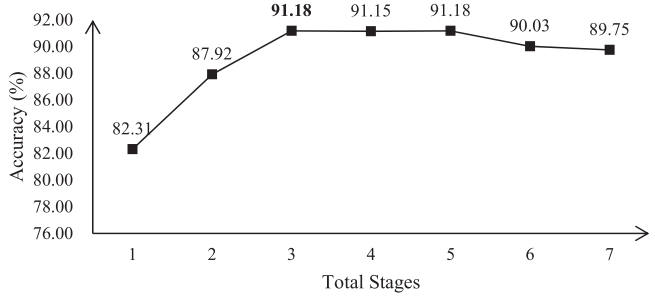


Fig. 14. Comparison across each number of total stages.

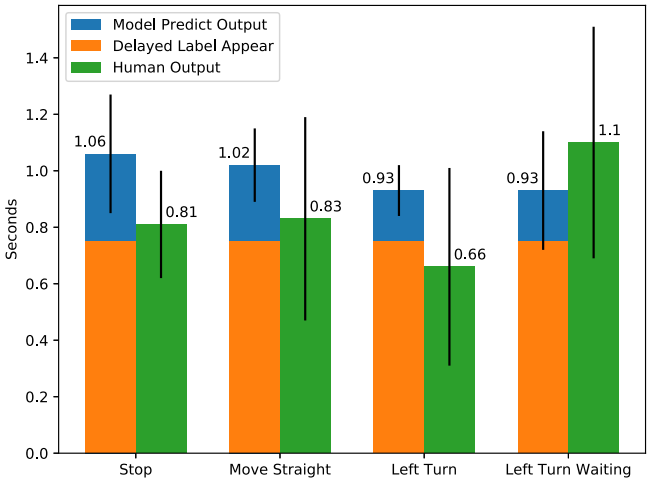


Fig. 15. Response speed comparison, gesture (1)–(4).

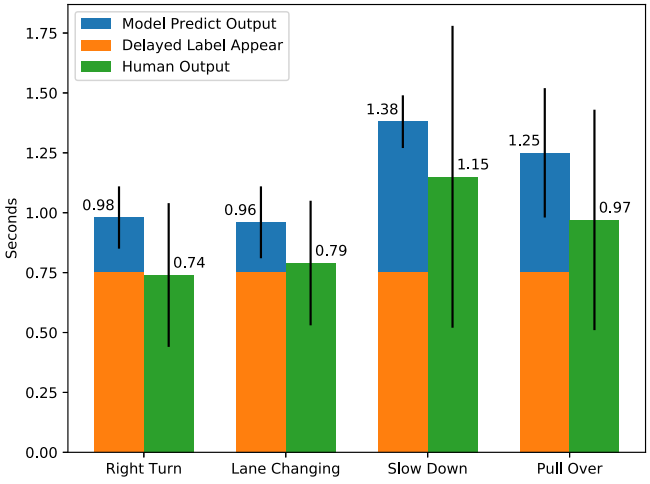


Fig. 16. Response speed comparison, gesture (5)–(8).

According to [1], adding more stages to the network results in better part detection rate. However, each stage adds approximately 14ms latency to the prediction process. considering the trade-off between the recognition accuracy and the latency, see Fig. 14, we decide to add 3 stages into our model.

We implement some typical network architectures for the police gesture recognizing task. A comparison of the architecture and accuracy between our method and other methods is listed in Table 3.

In Table 3, top 4 architecture extracted skeletal data, while other architectures use image features directly. The performance of bidirectional LSTM in architecture no.2 is close to LSTM, but since it needs to read information from future time points, the method can only execute on off-line data instead of online data. Architecture no.3 and no.4 use different connections than PKEN between convolutional layers. To achieve similar accuracy, these 2 architecture consume more times than PKEN. Architecture no.5 and no.6 classify the gestures from image features directly, which results in lower accuracy.

The appearances of training labels are delayed to increase the accuracy of model. However, this paper describes an online method that needs to recognize gestures as soon as possible, so the response speed must be in an acceptable range. To address this problem, a comparison between the performance of human and our method is presented as Figs. 15 and 16.

Figs. 15 and 16 compare the average response speed of our method with human. The response speed is defined by the time duration between the appearance of the original label and the appearance of the predicted label. In these figures, the horizontal axes represent eight different gestures, the vertical axes represent response speed by seconds. At the 0.00 second, the original labels of gestures starts to appear. The green bars represent the response speed of human. To acquire these data, videos in the test set of the police gesture dataset are played at normal speed, and 8 subjects with Chinese driving licences are asked to press a button as soon as they recognized a gesture. For each gesture, the response

speed of a driver is computed as the time the button was pressed minus the time the original label appeared. The final human response speed for each class of gesture is calculated as the average response speed of test samples. The orange bars indicate the appearance timing of delayed labels when training, their length are fixed at 0.75. Ideally, the network is trained to show the class of gestures after 750 milliseconds, so the response speeds for each gestures should be 750 ms, as shown in orange bars. However, at test time, the network actually takes more time to recognize each gestures, and this increased duration is illustrated by the blue bars in the figure. Therefore, the length of the orange bar plus that of the blue bar, which is indicated by the number on the top of the blue bar, shows the response time of our method. The vertical black line centered at the top of blue and green bars indicates the standard deviation of the corresponding data.

Figs. 15 and 16 show that generally, human consumes less time to response a gesture than our method, and the method needs even more time (blue bar) than the delayed duration (orange bar) to recognize the gestures. But the method is more precise, it gives significantly less standard deviations than human. Generally speaking, the speed gap between human and method is not huge (0.28 seconds at most), and sometimes human perform worse according to the deviation. Also, in practice, it is expected for a driver to response the police after he saw a whole gesture, which is a duration of at least 2 s. Therefore, the response time of our method is within an acceptable range.

We shorten the CPM network to decrease the execution time of each frame for real-time detection and recognition, which is an optimization that does not relevant to the response time discussed above. Theoretically, if the execution time of each frame became longer, to keep the real-time performance, we can just decrease the amount of frames per second to compensate. However, this also decrease the spatial features received per second, which results in less reliable classification outputs. Under the graphic card environment of Nvidia GTX 1080 TI, our method runs at 17.2 fps per second (we limited the speed at 15 fps for stability), receiving 15 frames of spatial features within a second. As a comparison, if original CPM is applied, the whole program runs at 8.7 fps, which receives significantly less spatial features (8–9 frames per second) when handling online tasks. Using ResNet as articulated pose extractor can further decrease this to 5.1 fps. The lack of frames can ultimately cause more classification errors to the online tasks. In summary, our method manage to achieve real-time performance with relatively more fps so as to keep a high accuracy when working online.

7. Conclusion and future work

In this paper, we integrate Convolutional Pose Machine with two kinds of handcrafted features: Relative Bone Length and Angle with Gravity, and adopt a LSTM network to capture the temporal domain feature and recognize police gestures accurately. Our method also provides an architecture that is able to perform online police gesture recognition tasks with high accuracy. The method extracts skeletal data from image by a modified CPM to decrease the influence of background scenes. Then, it converts the skeletal data to two kinds of handcrafted spatial features: Relative Bone Length and Angle with Gravity to avoid the noises from absolute positions. At last, it sends the sequence of spatial features to LSTM network for the temporal features, and finally uses a dense connected network to infer the class of gestures.

Future work will concentrate on recognizing traffic police gestures in multi-person scenes, and separating the traffic police from the crowd. This task includes recognizing multiple traffic police attire, and picking all the parts that belong to the police from a crowded scene. The dataset will be expanded as well to build the recognition model.

Declaration of Competing Interest

We have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] S.-E. Wei, V. Ramakrishna, T. Kanade, Y. Sheikh, Convolutional pose machines, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 4724–4732.
[2] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[3] B. Wang, T. Yuan, Traffic police gesture recognition using accelerometer, in: Proceedings of IEEE SENSORS Conference, Lecce, Italy, 2008, pp. 1080–1083.
[4] T. Yuan, B. Wang, Accelerometer-based Chinese traffic police gesture recognition system, Chin. J. Electron. 19 (2) (2010) 270–274.
[5] Z. Cai, F. Guo, Max-covering scheme for gesture recognition of Chinese traffic police, Pattern Anal. Appl. 18 (2) (2015) 403–418.
[6] F. Guo, J. Tang, Z. Cai, Automatic recognition of Chinese traffic police gesture based on max-covering scheme, Adv. Inf. Sci. Serv. Sci. 5 (1) (2013) 428.
[7] M. Eichner, V. Ferrari, Human pose co-estimation and applications, IEEE Trans. Pattern Anal. Mach. Intell. 34 (11) (2012) 2282–2288.
[8] M. Eichner, M. Marin-Jimenez, A. Zisserman, V. Ferrari, 2d articulated human pose estimation and retrieval in (almost) unconstrained still images, Int. J. Comput. Vis. 99 (2) (2012) 190–214.
[9] H. Kang, C.W. Lee, K. Jung, Recognition-based gesture spotting in video games, Pattern Recognit. Lett. 25 (15) (2004) 1701–1714.
[10] Y. Song, D. Demirdjian, R. Davis, Tracking body and hands for gesture recognition: natops aircraft handling signals database, in: Proceedings of 2011 IEEE International Conference on Automatic Face & Gesture Recognition and Workshops, IEEE, 2011, pp. 500–506.
[11] Z. Zhang, Microsoft kinect sensor and its effect, IEEE Multimed. 19 (2) (2012) 4–10.
[12] F. Guo, J. Tang, X. Wang, Gesture recognition of traffic police based on static and dynamic descriptor fusion, Multimed. Tools Appl. 76 (6) (2017) 8915–8936.
[13] Q.K. Le, C.H. Pham, T.H. Le, Road traffic control gesture recognition using depth images, IEIE Trans. Smart Process. Comput. 1 (1) (2012) 1–7.
[14] Z. Zhou, S. Li, B. Sun, Extreme learning machine based hand posture recognition in color-depth image, in: Proceedings of Chinese Conference on Pattern Recognition, Springer, 2014, pp. 276–285.
[15] M.H. Memon, J.-P. Li, I. Memon, Q.A. Arain, Geo matching regions: multiple regions of interests using content based image retrieval based on relative locations, Multimed. Tools Appl. 76 (14) (2017) 15377–15411.
[16] I. Memon, L. Chen, A. Majid, M. Lv, I. Hussain, G. Chen, Travel recommendation using geo-tagged photos in social media for tourist, Wirel. Pers. Commun. 80 (4) (2015) 1347–1362.
[17] Y. Bengio, A. Courville, P. Vincent, Representation learning: a review and new perspectives, IEEE Trans. Pattern Anal. Mach. Intell. 35 (8) (2013) 1798–1828.
[18] Z. Cao, T. Simon, S.-E. Wei, Y. Sheikh, Realtime multi-person 2d pose estimation using part affinity fields, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 7291–7299.
[19] D. Mehta, S. Sridhar, O. Sotnychenko, H. Rhodin, M. Shafiei, H.-P. Seidel, W. Xu, D. Casas, C. Theobalt, Vnect: Real-time 3d human pose estimation with a single rgb camera, ACM Trans. Graph. 36 (4) (2017) 44.
[20] H. Bilen, B. Fernando, E. Gavves, A. Vedaldi, S. Gould, Dynamic image networks for action recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 3034–3042.
[21] K. Soomro, A.R. Zamir, M. Shah, A dataset of 101 human action classes from videos in the wild, Center for Research in Computer Vision (2012).
[22] H. Kuehne, H. Jhuang, R. Stiefelhagen, T. Serre, Hmdb51: a large video database for human motion recognition, in: High Performance Computing in Science and Engineering, 12, Springer, 2013, pp. 571–582.
[23] J. Donahue, L. Anne Hendricks, S. Guadarrama, M. Rohrbach, S. Venugopalan, K. Saenko, T. Darrell, Long-term recurrent convolutional networks for visual recognition and description, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 2625–2634.
[24] B. Zhang, L. Wang, Z. Wang, Y. Qiao, H. Wang, Real-time action recognition with enhanced motion vector cnns, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 2718–2726.
[25] F. Zhu, L. Shao, J. Xie, Y. Fang, From handcrafted to learned representations for human action recognition: a survey, Image Vis. Comput. 55 (2016) 42–52.
[26] D. Wu, L. Shao, Leveraging hierarchical parametric networks for skeletal joints based action segmentation and recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2014, pp. 724–731.
[27] Y. Du, W. Wang, L. Wang, Hierarchical recurrent neural network for skeleton based action recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 1110–1118.
[28] B. Liu, H. Cai, Z. Ju, H. Liu, Rgb-d sensing based human action and interaction analysis: a survey, Pattern Recognition 94 (2019) 1–12.

[29] J.K. Aggarwal, L. Xia, Human activity recognition from 3d data: a review, Pattern Recognit. Lett. 48 (2014) 70–80.
[30] V. Argyriou, M. Petrou, S. Barsky, Photometric stereo with an arbitrary number of illuminants, Comput. Vis. Image Underst. 114 (8) (2010) 887–900.
[31] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE Conference on Computer vision and Pattern Recognition, 2015, pp. 1–9.
[32] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, J. Mach. Learn. Res. 15 (1) (2014) 1929–1958.
[33] J. Wu, H. Zheng, B. Zhao, Y. Li, B. Yan, R. Liang, W. Wang, S. Zhou, G. Lin, Y. Fu, et al., Large-scale datasets for going deeper in image understanding, 2019 IEEE International Conference on Multimedia and Expo (ICME), 2019, pp. 1480–1485.
[34] L. Pigou, A. Van Den Oord, S. Dieleman, M. Van Herreweghe, J. Dambre, Beyond temporal pooling: Recurrence and temporal convolutions for gesture recognition in video, Int. J. Comput. Vis. 126 (2-4) (2018) 430–439.
[35] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[36] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks., in: Proceedings of CVPR, 1, 2017, p. 3.
[37] S. Ji, W. Xu, M. Yang, K. Yu, 3d convolutional neural networks for human action recognition, IEEE Trans. Pattern Anal. Mach. Intell. 35 (1) (2013) 221–231.
[38] S. Xingjian, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, W.-c. Woo, Convolutional lstm network: a machine learning approach for precipitation nowcasting, in: Proceedings of Advances in Neural Information Processing Systems, 2015, pp. 802–810.
[39] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, in: Proceedings of the IEEE International Conference on Computer Vision, 2015, pp. 1026–1034.
[40] D.P. Kingma, J. Ba, Adam: A Method for Stochastic Optimization, arXiv:1412.6980 (2014).
[41] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the Thirteenth International Conference on Artificial intelligence and Statistics, 2010, pp. 249–256.
[42] S.-F. Wong, R. Cipolla, Continuous gesture recognition using a sparse Bayesian classifier, in: Proceedings of 18th International Conference on Pattern Recognition, ICPR, 1, IEEE, 2006, pp. 1084–1087.



Jian He received the M.S. degree in Computer Software from Northwest University, Xin, China in 2000, and received the Ph.D. degree in Computer Software from Xin Jiaotong University, Xin, China 2005. He is an associate professor at the School of Software Engineering, Beijing University of Technology. Besides, he is also an adjunct researcher with Beijing Advanced Innovation Center for Future Internet Technology, China. His research interests include Ubiquitous Computing, Embedded System, and HCI.



Cheng Zhang He received the B.S. degree in Software Engineering from Beijing University of Technology in 2016. He is a graduate student at the Information Department, Beijing University of Technology. His research interests include Computer Vision and Deep Learning.



Xinlin He is an undergraduate student at The University of British Columbia studying Computer Science. She is interested in data analytics, operational research, and computer graphics.



Ruihai Dong is an Assistant Professor with the School of Computer Science in University College Dublin. His research interests lie broadly in Machine Learning and Deep Learning, and their applications in recommender systems and finance.